

Graph-Based Molecular Generation via Adversarial Learning with Hierarchical Discriminators and Reward-Driven Objectives

Kaleb Ray, Ethan Triem, Seth Kacura, Aaron Sierra, David Day, Cheyenne Garza

Baylor University

Waco, TX, United States

{kaleb_ray1, ethan_triem2, seth_kacura1, aaron_sierra1, david_day1, cheyenne_garza1}@baylor.edu

Abstract—Recent advances in deep-generative models have allowed the generation of chemically valid and novel molecules with desirable properties for further investigation. In this work, we provide such a model, integrating generative adversarial networks (GANs), hierarchical graph critic, and reinforcement learning. Our graph-based generator generates molecular graphs in the form of adjacency and node feature matrices. To improve robustness and stability during training, we implement a hierarchical differentiable pooling (DIFFPOOL)-based discriminator. This type of discriminator allows our model to learn coarse-grained graph representations during training. Additionally, we introduce a reinforcement learning component in which a policy network receives reward signals based on predicted chemical properties, guiding the generator toward structures with high drug-likeness and other target metrics. In evaluating our approach, we look for validity, uniqueness, novelty, and alignment with drug-likeness metric scores. The results demonstrate improved generation quality and training stability compared to baseline graph GANs, particularly when incorporating DIFFPOOL and reinforcement-guided reward shaping. Our findings suggest that the combination of hierarchical discrimination with reinforcement learning shows a positive trajectory for property-driven molecular design in drug discovery and materials science.

Index Terms—GAN, Hierarchical discriminator, DIFFPOOL, Reinforcement learning, Validity, Uniqueness, Novelty, Drug-likeness

Nomenclature

$A \in \mathbb{R}^{n \times n}$	adjacency matrix
$B \in \mathbb{Z}^{n \times n}$	bond order matrix (optional)
$G = (V, E)$	molecular graph with vertices V and edges E
$z \in \mathbb{R}^d$	latent vector
n	number of nodes (atoms)
f	number of node features (atom types)
$A \in [0, 1]^{n \times n}$	soft adjacency matrix
$T \in \mathbb{R}^{n \times f}$	node feature matrix (softmax over atom types)
$T_i \in \mathbb{R}^f$	one-hot (or softmax) encoded atomic type of node v_i
$\mathcal{A}(z)$	flattened adjacency matrix
$\mathcal{T}(z)$	flattened node features output
$\mathcal{H} = v_i \in V : t_i = 1$	set of hydrogen atoms
$R(M)$	total reward for generated molecule M
$R \leftarrow \alpha R + (1 - \alpha)R(M), \alpha = 0.99$	exponentially smoothed baseline reward
$\lambda_G = 3.0, \lambda_R = 7.0$	scaling factors for generator and reward loss in scaled loss computation
$x^{(0)} = T$	input node features
$\sigma(\cdot)$	activation function
$\hat{y} \in \mathbb{R}^{n \times 1}$	score per node critic output
$S^{(1)} \in \mathbb{R}^{n \times a_1}, S^{(2)} \in \mathbb{R}^{n \times a_2}$	assignment matrices for the two DiffPool layers (with $a_1 = 10, a_2 = 5$ by default)
$Z^{(1)} \in \mathbb{R}^{n \times d_1}, Z^{(2)} \in \mathbb{R}^{a_1 \times d_2}$	node embeddings after each layer ($d_1 = 64, d_2 = 128$)

I. INTRODUCTION

In our research, we used the QM9 dataset to train a GAN generator working alongside a hierarchical discriminator to generate chemically valid and novel molecules for scientific study. We used two different versions of the hierarchical discriminator, one with DIFFPOOL, and one without. In association with the generator and discriminator, we also implement a reward policy network to encourage the model’s generation. The dataset we used, QM9, contained the following variables: mol_id, smiles, A, B, C, mu, alpha, homo, lumo, gap, r2, zpve, u0, u298, h298, g298, cv, u0_atom, u298_atom, h298_atom, g298_atom, Morgan_fingerprint, adj_matrix, node_features, and node_count. The variables that are significant for this work are adj_matrix, node_features, and node_count. These variables define the molecules that are fed into the generator

during training. The adjacency matrix, `adj_matrix`, contains information on which bonds are present inside the molecule. The node features, `node_features`, contain information on which atoms are present within the molecule. In this study, we analyze the validity, novelty, uniqueness, and drug-likeness of molecules produced by the generator. The validity is based on whether or not the generated molecule is chemically valid, uniqueness is based on similarity of the generated molecule to previous molecules, novelty is determined by whether or not a similar molecule is already present within the training data, and drug-likeness is concerned with the similarity of the molecule generated to common drug-like molecules.

II. LITERARY REVIEW

In recent years, AI-driven methods have gained traction in *de novo* drug design, offering the potential to generate novel chemical compounds without reliance on predefined molecular databases. These methods offer the potential to eliminate the dependence on existing compound databases and accelerate the drug discovery pipeline with optimized pharmacological properties.

Among the most promising machine learning tools applied to molecular generation are graph-based generative models. Unlike traditional representations such as SMILES (Simplified Molecular Input Line Entry System) strings, graph-based models treat molecules as undirected graphs, where atoms are represented as nodes and chemical bonds as edges. This approach more accurately captures molecular structure and allows for more chemically valid generation processes. Within this domain, Graph Neural Networks (GNNs) and Generative Adversarial Networks (GANs) have become foundational model architectures.

MolGAN, introduced by De Cao and Kipff[1], represents one of the earliest applications of GANs to molecular graph generation. It combines an implicit generative model with a reinforcement learning objective to generate molecular graphs that are both valid and optimized for specific chemical properties. The model eliminates the need for graph matching or predefined node orders by using an adjacency matrix representation of molecules. Despite demonstrating high chemical validity, MolGAN suffers from limited output diversity due to mode collapse—a common issue in GAN-based models—and lacks structural regularization mechanisms.

To address these limitations, Guarino et al. [2] proposed DiPol-GAN, an extension of MolGAN that introduces two key innovations. First, it incorporates Differentiable Pooling (DIFFPOOL), a hierarchical pooling mechanism that enables the network to learn coarse-grained representations of molecular graphs. This facilitates hierarchical feature extraction and better captures the global context of molecular structures. Second, the model integrates a reinforcement learning policy gradient framework to further optimize molecular generation based on property-specific reward functions. DiPol-GAN has shown improvements in the generation of property-compliant and structurally diverse molecules, however, it also introduces

additional complexity in training and higher computational overhead.

Several alternative architectures have also been explored in the literature. For instance, GraphAF, considered by Shi et al.[6], employs autoregressive flow-based modeling to sequentially construct molecular graphs in a chemically valid manner. While this ensures structural integrity during generation, the method can be computationally intensive and less suited to multi-objective optimization. Similarly, latent variable models such as GraphDF, authored by Luo et. Al[7], attempt to decouple the representation and property optimization process, offering greater interpretability and flexibility, though often at the cost of sample efficiency.

Despite these advancements, open challenges persist. The generation of molecules that are simultaneously valid, diverse, and property-optimized remains non-trivial. Many models struggle with achieving all three objectives concurrently. Additionally, evaluation metrics in molecular generation are often inconsistent, making it difficult to directly compare model performance across studies. Reinforcement learning-based optimization strategies, while effective in fine-tuning molecular properties, can further complicate training stability and scalability.

In this context, the present study revisits the DiPol-GAN architecture to evaluate its practical utility under modern evaluation standards and datasets. By reproducing the original model and conducting systematic benchmarking, we aim to assess its ability to generate chemically valid and functionally relevant molecular graphs. Furthermore, we explore potential enhancements to DiPol-GAN’s training protocol and reward shaping functions to improve output diversity and property alignment. This work contributes toward validating and extending graph-based generative approaches in *de novo* drug discovery.

III. METHODS

A. Dataset and Preprocessing

We trained our model using the QM9 dataset [?], a collection of over 134,000 small organic molecules composed of hydrogen, carbon, nitrogen, oxygen, and fluorine. Each molecule was transformed into a graph structure using RDKit, resulting in an adjacency matrix $A \in \mathbb{R}^{n \times n}$ and a node feature matrix $T \in \mathbb{R}^{n \times f}$, where n is the number of atoms and f is the number of atom types. We filtered the data to exclude overly large molecules and standardized each molecule to a maximum node count of 9 and 5 node features (atom types). SMILES strings were retained for uniqueness and novelty evaluation.

B. Generator Architecture

The generator is a neural network that maps a latent vector $z \in \mathbb{R}^d$ to a molecular graph in the form of (A, T) , as described in Eq. (??). Internally, the generator uses a multi-layer perceptron to output a flattened adjacency matrix and node feature matrix, which are then reshaped, symmetrized, and softmax-normalized. We used soft Gumbel-Softmax sampling

to approximate discrete outputs. Atom types are then clipped and mapped to valid atomic numbers: H, C, N, O, and F.

The generator is trained using a combined loss function consisting of binary cross-entropy (Eq. (??)) and a reward-weighted policy gradient component based on total reward $R(M)$ (Eq. (??)). Training uses the Adam optimizer with a learning rate of 0.0005 and $\beta_1 = 0.5$.

C. Discriminator Architectures

We implemented two versions of the discriminator: one that processes node features directly and another that incorporates global pooling to capture graph-level representations.

1) *Without Global Pooling*: The baseline discriminator processes the node features using two fully connected layers with ReLU activations, followed by a sigmoid output layer (see Eq. (??)). It is trained using binary cross-entropy loss with label smoothing for real and fake labels (Eq. (??)).

2) *With Global Pooling*: The second discriminator variant extends the baseline by adding a global average pooling layer after the dense layers, which aggregates node-level representations into a single graph embedding before classification. This allows the model to capture coarse structural information without explicitly learning assignment matrices, as in DiffPool [?].

D. Reward Policy Network

We used reinforcement learning to guide generation toward chemically meaningful and pharmaceutically desirable structures. The reward function $R(M)$ consists of five components:

- **Validity**: Ensures chemical sanitization (Eq. (??)).
- **Stray Hydrogens**: Penalizes unconnected hydrogen atoms (Eq. (??)).
- **Uniqueness**: Encourages new molecules within a generation batch (Eq. (??)).
- **Novelty**: Rewards molecules not seen in training (Eq. (??)).
- **Drug-likeness**: Uses RDKit’s QED score (Eq. (??)).

Rewards are combined and smoothed using a running baseline $\bar{R}$ to compute the advantage (Eq. (??)), which is used in the generator’s scaled loss (Eq. (??)).

E. Training Protocol

We trained our models for up to 10 epochs using batch sizes of 16. Latent vectors z were sampled from a standard Gaussian distribution. For the DiffPool version, the discriminator used 64 hidden units and clustered nodes into 5 groups, as described in Eq. (??). All experiments used the Adam optimizer with a learning rate of 1×10^{-4} .

F. Evaluation Metrics

Model performance was evaluated on four core metrics:

- **Validity**: Percentage of generated molecules that passed sanitization.
- **Uniqueness**: Fraction of distinct molecules generated in a batch.

- **Novelty**: Fraction of generated molecules not found in the training set.
- **Drug-likeness**: Average QED score over valid molecules.

Metrics were computed using RDKit and compared across model variants to assess the impact of DiffPool and reward shaping.

IV. EQUATIONS

A. Adjacency Matrix to Molecule

1) *Vertex Set (Atoms)*: Each atom is represented by a vertex:

$$V = \{v_i = \text{Atom}(t_i) | i = 1, 2, \dots, n\} \quad (1)$$

Optionally, if chirality tags are given:

$$\text{Chiral}(v_i) = c_i \text{ for } i \in C \subseteq \{1, \dots, n\} \quad (2)$$

2) *Edge Set (Bonds)*: For all $i < j$:

If $A_{ij} > \tau$:

$$o_{ij} = \begin{cases} B_{ij}, & \text{if } B_{ij} > 0 \\ 1, & \text{otherwise} \end{cases}$$

$$E \ni e_{ij} = (v_i, v_j, \beta(o_{ij}))$$

Where $\beta(o)$ maps bond order to bond type:

$$\beta(o) = \begin{cases} \text{SINGLE}, & o = 1 \\ \text{DOUBLE}, & o = 2 \\ \text{TRIPLE}, & o = 3 \\ \text{AROMATIC}, & o = 4 \\ \text{undefined}, & \text{otherwise} \end{cases} \quad (3)$$

3) *Isolated Hydrogen Handling*: Let:

$$H = i | t_i = 1 \wedge \deg(v_i) = 0 \quad (4)$$

For each $h \in H$:

$$j^* = \arg \max_{\substack{j \notin H \\ j \neq h}} A_{hj} \quad (5)$$

If $\deg(v_{j^*}) < \text{Valence}_{\max} t_{j^*}$, then:

$$E \ni e_{hj^*} = (v_h, v_{j^*}, \text{SINGLE}) \quad (6)$$

Else:

$$V \leftarrow V \setminus \{v_h\} \quad (7)$$

4) *Molecule*: The final molecule is:

$$M = \text{Sanitize}(G = (V, E)) \quad (8)$$

B. Reward Policy Network

1) *Validity Reward*: Let M be a candidate molecule derived from (A, T) . Let $\text{Valence}_{\max}(t_i)$ be the maximum valence for atom type t_i , and let $\deg(v_i)$ be the current explicit valence (degree) of atom $v_i \in V$.

$$R_{\text{validity}}(M) = \begin{cases} -0.5, & \text{if } M \text{ is None or fails sanitization} \\ \max(0, 5 - |\sum_{v_i \in V} (\text{Valence}_{\max}(t_i) - \deg(v_i))|), & \text{otherwise} \end{cases} \quad (9)$$

2) *Stray Hydrogen Reward*: Let $H \subset V$ be the set of hydrogen atoms, and let $\deg(v_i)$ be the degree of atom v_i .

$$R_{\text{stray}}(M) = \begin{cases} -0.5, & \text{if } M \text{ is None or } \exists v_i \in H \text{ such that } \deg(v_i) = 0 \\ 0.5, & \text{otherwise} \end{cases} \quad (10)$$

3) *Uniqueness Reward*: Let $\mathcal{G} = \{S_1, S_2, \dots, S_n\}$ be the list of SMILES strings for previously generated molecules, and let $S(M)$ be the SMILES of the current molecule M . Let $\text{count}_{\mathcal{G}}(S(M))$ be the number of times $S(M)$ appears in $\mathcal{G}$.

$$R_{\text{unique}}(M, \mathcal{G}) = \begin{cases} -0.3, & \text{if } R_{\text{validity}}(M) = 0 \\ \max\left(0, \min\left(1, 1 - \frac{|G|+1}{\text{count}_{\mathcal{G}}(S(M))}\right)\right), & \text{otherwise} \end{cases} \quad (11)$$

4) *Novelty Reward*: Let $\mathcal{T} \subset \mathbb{S}$ be the set of training SMILES strings.

$$R_{\text{novelty}}(M, T, \mathcal{G}) = \begin{cases} -0.3, & \text{if } R_{\text{validity}}(M) = 0 \\ -2.5, & \text{if } R_{\text{unique}}(M, \mathcal{G}) < 1 \\ -0.7, & \text{if } S(M) \in T \\ 2.3, & \text{otherwise} \end{cases} \quad (12)$$

5) *Drug-likeness Reward*: Let $Q(M) \in [0, 1]$ be the RDKit QED score of molecule M .

$$R_{\text{drug}}(M) = \begin{cases} Q(M), & \text{if } R_{\text{validity}}(M) > 0 \\ -0.3, & \text{otherwise} \end{cases} \quad (13)$$

6) *Total Reward*:

$$R_{\text{validity}}(M) + R_{\text{stray}}(M) + R_{\text{unique}}(M, \mathcal{G}) + R_{\text{novelty}}(M, T, \mathbb{G}) + R_{\text{drug}}(M) \quad (14)$$

C. Generator

1) *Generator Function*: The generator is a function:

$$G: \mathbb{R}^d \rightarrow ([0, 1]^{n \times n}, \mathbb{R}^{n \times f}) \quad (15)$$

constructed from a multi-layer perceptron (MLP):

$$\text{MLP}(z) = \mathcal{A}(Z) \parallel \mathcal{T}(z) \quad (16)$$

Where:

- $\mathcal{A}(z) \in \mathbb{R}^{n^2}$, reshaped and symmetrized:

$$A = \frac{1}{2}(\text{reshape}(\mathcal{A}(z)) + (\text{reshape}(\mathcal{A}(z))^T) \quad (17)$$

- Then row-wise softmax is applied to normalize:

$$A_{ij} = \frac{\exp(A_{ij})}{\sum_k \exp(A_{ik})} \quad (18)$$

- T is obtained from:

$$T = \text{reshape}(\mathcal{T}(z)) \in \mathbb{R}^{n \times f}, T_i = \text{softmax}(T_i) \quad (19)$$

- Then the atom types $t_i \in 1, 6, 7, 8, 9$ are selected by:

$$t_i = \beta(\arg \max_k T_{ik}) \quad (20)$$

Where $\beta(o)$ is the bond type mapping (i.e., atomic number index map).

2) *Hydrogen Fixing Logic*: For each $v_i \in \mathcal{H}$, if:

$$\deg(v_i) = \sum_j A_{ij} < \tau \text{ with } \tau = 0.5 \quad (21)$$

Then set:

$$A_{ij^*} = A_{j^*i} = 0.9 \text{ where } j^* = \arg \max_{j \neq i} A_{ij} \quad (22)$$

3) *Critic Loss*: Let D be the discriminator. For real and generated pairs:

- $D(A, T) \in [0, 1]$
- Use smoothed labels: $y_{\text{real}} = 0.9, y_{\text{fake}} = 0.1$

Binary cross-entropy:

$$\mathcal{L}_D = \text{BCE}(y_{\text{real}}, D(A_{\text{real}}, T_{\text{real}})) + \text{BCE}(y_{\text{fake}}, D(A_{\text{gen}}, T_{\text{gen}})) \quad (23)$$

4) *Generator Loss with Reinforcement Learning*: Advantage:

$$A(M) = R(M) - \bar{R} \quad (24)$$

Generator loss:

$$\mathcal{L}_G = \text{BCE}(1, D(A_{\text{gen}}, T_{\text{gen}})) \quad (25)$$

Combined scaled loss:

$$\mathcal{L}_{\text{scaled}} = \lambda_G \times \mathcal{L}_G - \lambda_R \times \mathbb{E}_{z \sim \mathcal{N}(0,1)} [\log(P(z)) \times A(M)] \quad (26)$$

D. Critic without Differentiable Pooling

1) *First Layer (Dense Transformation)*:

$$x^{(1)} = \text{ReLU}(W_1 x^{(0)} + b_1), W_1 \in \mathbb{R}^{f \times 64} \quad (27)$$

2) *Second Layer (Dense Transformation)*:

$$x^{(2)} = \text{ReLU}(W_2 x^{(1)} + b_2), W_2 \in \mathbb{R}^{64 \times 128} \quad (28)$$

3) *Final Layer (Output Scores)*:

$$\hat{y} = \sigma(W_3 x^{(2)} + b_3), W_3 \in \mathbb{R}^{128 \times 1} \quad (29)$$

E. Critic with Differentiable Pooling

1) *Differentiable Pooling Layer 1*: Assignment matrix and node embeddings:

$$S^{(1)} = \text{softmax}(TW_s^{(1)} + b_s^{(1)}) \in \mathbb{R}^{n \times a_1} \quad (30)$$

$$Z^{(1)} = \text{ReLU}(TW_z^{(1)} + b_z^{(1)}) \in \mathbb{R}^{n \times d_1} \quad (31)$$

Graph coarsening (pooled features and adjacency):

$$T^{(1)} = X^{(1)} = (S^{(1)})^T Z^{(1)} \in \mathbb{R}^{a_1 \times d_1} \quad (32)$$

$$A^{(1)} = (S^{(1)})^T A S^{(1)} \in \mathbb{R}^{a_1 \times a_1} \quad (33)$$

2) *Differentiable Pooling Layer 2*:

$$S^{(2)} = \text{softmax}(T^{(1)} W_s^{(2)} + b_s^{(2)}) \in \mathbb{R}^{a_1 \times a_2} \quad (34)$$

$$Z^{(2)} = \text{ReLU}(T^{(1)} W_z^{(2)} + b_z^{(2)}) \in \mathbb{R}^{a_1 \times d_2} \quad (35)$$

$$T^{(2)} = X^{(2)} = (S^{(2)})^T Z^{(2)} \in \mathbb{R}^{a_2 \times d_2} \quad (36)$$

$$A^{(2)} = (S^{(2)})^T A^{(1)} S^{(2)} \in \mathbb{R}^{a_2 \times a_2} \quad (37)$$

3) *Global Pooling and Classification*: Global average pooling over node features:

$$\bar{T} = \frac{1}{a_2} \sum_{i=1}^{a_2} T_i^{(2)} \in \mathbb{R}^{d_2} \quad (38)$$

Final prediction (binary classification):

$$\hat{y} = \sigma(\bar{T}^T w + b) \in [0, 1] \quad (39)$$

REFERENCES

- [1] N. De Cao and T. Kipf, "MolGAN: An implicit generative model for small molecular graphs," *arXiv preprint arXiv:1805.11973*, 2018.
- [2] Rivas, P., Guarino, M., Shah, A. "DiPol-GAN: Generating Molecular Graphs Adversarially with Relational Differentiable Pooling". *NeurIPS Workshop* (2019).
- [3] Ying, Z., You, J., Morris, C., Ren, X., Hamilton, W., Leskovec, J. (2018). Hierarchical Graph Representation Learning with Differentiable Pooling. *Advances in Neural Information Processing Systems*, 4800–4810.
- [4] Ramakrishnan, R., Dral, P. O., Rupp, M., von Lilienfeld, O. A. (2014). Quantum Chemistry Structures and Properties of 134 Kilo Molecules. *Scientific Data*, 1. (QM9)
- [5] ZINC Database – <http://zinc15.docking.org/> (ZINC-250k)
- [6] W. Shi, M. Xu, Z. Zhu, W. Zhang, M. Zhang, J. Tang "GraphAF: A flow-based autoregressive model for molecular graph generation," in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2020
- [7] Y. Liu, K. Yan, S. HJi, "GraphDF: A flow-based autoregressive model for molecular graph generation," in *Proc. Advances in Neural Inf. Process. Syst. (NeurIPS)*, 2021.

IEEE conference templates contain guidance text for composing and formatting conference papers. Please ensure that all template text is removed from your conference paper prior to submission to the conference. Failure to remove the template text from your paper may result in your paper not being published.